Algorithmique 1 : méthodologie de la programmation impérative
Contrôle continu n° 1

Date : 07-03-2025. Durée : 1 h 30. Documents, casques audio, oreillettes, calculatrices, mobiles, tablettes, portables, objets connectés, montres, trousses... interdites.

Commencez par renseigner TOUTES les cases « NOM Prénom numéro (d'étudiant·e) » qui figurent sur la liasse ; il y en a TROIS. Toute question attend une réponse dans le cadre situé immédiatement sous elle. En cas de manque de place : il sera écrit dans la case « VOIR COPIE À PART » ; la réponse, précédée de son numéro, sera donnée dans son intégralité sur une copie à part, fournie à la demande ; toute copie à part utilisée ne pourra l'être que pour un exercice donné. Ni l'espace figurant à l'intérieur des cadres ni les copies ne sauraient faire office de brouillon : APPLIQUEZ-VOUS ; N'ÉCRIVEZ PAS AU CRAYON À PAPIER..

NOM Prénom numéro :

Exercice 1 *(3 points.)*

Q1 Qu'a-t-on montré lorsque l'on a établi la correction partielle d'un programme ou d'un algorithme ?

Q2 Que reste-t-il à montrer lorsque l'on n'a établi que la correction partielle d'un programme ou d'un algorithme ?

Q3 Qu'est-ce qu'une assertion dans le contexte de la logique de Hoare ?

Q4 Qu'est-ce qu'une quantité de contrôle ?

Exercice 2 *(3 points.)*

Pour chacune des propositions suivantes : **a)** dites si elle est vraie (VRAI) ou fausse (FAUX) ; **b)** démontrez ce que vous affirmez.

Q5 Pour tout entier n, pour tout réel x : $n \leqslant \lfloor x \rfloor \Rightarrow n \leqslant x$.

a)
b)

Q6 Pour tout entier n, pour tout réel x : $n \leqslant x \Rightarrow n \leqslant \lfloor x \rfloor$.

a)
b)

Q7 Pour tout entier n, pour tout réel x : $n \leqslant \lceil x \rceil \Rightarrow n \leqslant x$.

a)
b)

Q8 Pour tout entier n, pour tout réel x : $n \leqslant x \Rightarrow n \leqslant \lceil x \rceil$.

a)
b)

NOM Prénom numéro :

Exercice 3 *(4 points.)*

Chacune des séquences suivantes est formée d'une initialisation de la variable k d'un type entier et d'une boucle susceptible de modifier la valeur de k. Pour chacune d'elles : **a)** donnez LE PLUS FIN des invariants de boucle possibles, Q, qui porte sur la valeur de k et qui est satisfait après l'exécution de l'initialisation ; **b)** prouvez Q est bien un invariant de la boucle.

Indication : vous pouvez être amené à composer avec le reste de la division de la valeur de k par un entier à préciser.

Q9 $k \leftarrow 5$
tant que $k < 100$ **faire**
    $k \leftarrow k + 1$

a) Q :
b)

Q10 $k \leftarrow 20$
tant que $k < 25$ **faire**
    $k \leftarrow k - 1$

a) Q :
b)

Q11 $k \leftarrow 2$
tant que $k < 20$ **faire**
    $k \leftarrow k + 3$

a) Q :
b)

Q12 $k \leftarrow 0$
tant que $k < 2026$ **faire**
    si $k \bmod 2 = 0$ **alors**
        $k \leftarrow k + 3$
    sinon
        $k \leftarrow k + 1$

a) Q :
b)

NOM Prénom numéro :

Exercice 4 *(10 points.)*

Soit $B(x, p, n)$ le nombre défini pour tout réel x, tous entiers p et n tels que $0 \leqslant n \leqslant p$ par

$$B(x, p, n) = \sum_{j=0}^{n} \frac{p!}{j!(p-j)!} x^j.$$

Dans la suite vous devez donner des expressions de $B(x, p, n)$ pour certaines valeurs de p et n, puis construire un algorithme de calcul de $B(x, p, n)$ dans un modèle calculatoire où ne figurent pas les opérations élévation à la puissance et factorielle, puis porter cet algorithme en C.

Q13 Expression de $B(x, 4, 1)$.

Q14 Expression de $B(x, 4, 3)$.

Q15 Identificateurs des données de l'algorithme.

Q16 Identificateurs des variables de l'algorithme.

Q17 Assertion d'entrée P.

Q18 Assertion de sortie R.

Q19 Assertion Q (futur invariant de boucle). *Conseil :* utilisez « B » avec de bons paramètres.

Q20 Fin de l'induction : expression d'une condition (d'itération) C telle que $Q \wedge \neg C \Rightarrow R$.

Page intentionnellement vide.

Q21 Preuve que $Q \wedge \neg C \Rightarrow R$.

Q22 Induction (hérédité) : expression d'une action A' qui se termine et qui satisfait le schéma pré-post $Q \wedge C \xRightarrow{A'} Q$.

Q23 Preuve que $Q \wedge C \xRightarrow{A'} Q$.

Page intentionnellement vide.

Q24 Expression d'une quantité de contrôle pour la boucle associée au schéma pré-post $Q \wedge C \overset{A'}{\Longrightarrow} Q$.

Q25 Preuve que la quantité de contrôle annoncée en est bien une.

Q26 Base (initialisation) : expression d'une action A qui se termine et qui satisfait le schéma pré-post $P \overset{A}{\Longrightarrow} Q$.

Q27 Expression, sans la paire d'accolades englobantes, du corps d'une fonction C, de type résultant **`double`**, de paramètres `x` du type **`double`**, `p` et `n` du type **`int`**, qui renvoie $B(\texttt{x}, \texttt{p}, \texttt{n})$ lorsque ce nombre est défini et qui, sinon, donne à `errno` la valeur `EDOM` et renvoie une valeur nulle. Inutile de redonner l'invariant de boucle et la quantité de contrôle.

Corrigé

Solution à l'exercice 1

Q1 Qu'a-t-on montré lorsque l'on a établi la correction partielle d'un programme ou d'un algorithme ?

Voir support de cours et d'exercices.

Q2 Que reste-t-il à montrer lorsque l'on n'a établi que la correction partielle d'un programme ou d'un algorithme ?

Voir support de cours et d'exercices.

Q3 Qu'est-ce qu'une assertion dans le contexte de la logique de Hoare ?

Voir support de cours et d'exercices.

Q4 Qu'est-ce qu'une quantité de contrôle ?

Voir support de cours et d'exercices.

Solution à l'exercice 2

Pour chacune des propositions suivantes : **a)** dites si elle est vraie (VRAI) ou fausse (FAUX) ; **b)** démontrez ce que vous affirmez.

Q5 Pour tout entier n, pour tout réel x : $n \leqslant \lfloor x \rfloor \Rightarrow n \leqslant x$.

a) VRAI
b) C'est une conséquence immédiate de ce que $\lfloor x \rfloor \leqslant x$.

Q6 Pour tout entier n, pour tout réel x : $n \leqslant x \Rightarrow n \leqslant \lfloor x \rfloor$.

a) VRAI
b) $\lfloor x \rfloor$ est le plus grand des entiers $\leqslant x$. Or $n \leqslant x$. Il s'ensuit que $n \leqslant \lfloor x \rfloor$.

Q7 Pour tout entier n, pour tout réel x : $n \leqslant \lceil x \rceil \Rightarrow n \leqslant x$.

a) FAUX
b) Contre-exemple : $n = 1$, $x = 0{,}89$. Comme $\lceil 0{,}89 \rceil = 1$, on a $n \leqslant \lceil x \rceil$. Mais $n \nleqslant x$.

Q8 Pour tout entier n, pour tout réel x : $n \leqslant x \Rightarrow n \leqslant \lceil x \rceil$.

a) VRAI
b) C'est une conséquence immédiate de ce que $x \leqslant \lceil x \rceil$.

Solution à l'exercice 3

Chacune des séquences suivantes est formée d'une initialisation de la variable k d'un type entier et d'une boucle susceptible de modifier la valeur de k. Pour chacune d'elles : **a)** donnez LE PLUS FIN des invariants de boucle possibles, Q, qui porte sur la valeur de k et qui est satisfait après l'exécution de l'initialisation ; **b)** prouvez Q est bien un invariant de la boucle.

Indication : vous pouvez être amené à composer avec le reste de la division de la valeur de k par un entier à préciser.

Q9 $k \leftarrow 5$
tant que $k < 100$ **faire**
    $k \leftarrow k + 1$

a) $Q : 5 \leqslant k \leqslant 100$
b) $Q \wedge k < 100$
$\Rightarrow 5 \leqslant k \leqslant 99$
$\xRightarrow{k \leftarrow k+1} 6 \leqslant k \leqslant 100$
$\Rightarrow Q$

Q10 $k \leftarrow 20$
tant que $k < 25$ **faire**
    $k \leftarrow k - 1$

a) $Q : k \leqslant 20$
b) $Q \wedge k < 25$
$\Rightarrow k \leqslant 20$
$\xRightarrow{k \leftarrow k-1} k \leqslant 19$
$\Rightarrow Q$

Q11 $k \leftarrow 2$
tant que $k < 20$ **faire**
    $k \leftarrow k + 3$

a) $Q : 2 \leqslant k \leqslant 20 \wedge k \bmod 3 = 2$
b) $Q \wedge k < 20$
$\Rightarrow 2 \leqslant k \leqslant 19 \wedge k \bmod 3 = 2$
$\Rightarrow 2 \leqslant k \leqslant 17 \wedge k \bmod 3 = 2$
$\xRightarrow{k \leftarrow k+3} 5 \leqslant k \leqslant 20$
$\Rightarrow Q$

L'invariant de boucle suivant est correct mais moins fin :

a) $Q : 2 \leqslant k \leqslant 22$
b) $Q \wedge k < 20$
$\Rightarrow 2 \leqslant k \leqslant 19$
$\xRightarrow{k \leftarrow k+3} 5 \leqslant k \leqslant 22$
$\Rightarrow Q$

Q12

$k \leftarrow 0$
tant que $k < 2026$ **faire**
  si $k \bmod 2 = 0$ **alors**
    $k \leftarrow k + 3$
  sinon
    $k \leftarrow k + 1$

a) $Q : 0 \leqslant k \leqslant 2027 \wedge (k \bmod 4 = 0 \vee k \bmod 4 = 3)$
b) $Q \wedge k < 2026$
$\Rightarrow 0 \leqslant k \leqslant 2025 \wedge (k \bmod 4 = 0 \vee k \bmod 4 = 3)$
$\Rightarrow (0 \leqslant k \leqslant 2024 \wedge k \bmod 4 = 0) \vee (0 \leqslant k \leqslant 2023 \wedge k \bmod 4 = 3)$
$\xRightarrow{\textbf{si } k \bmod 2 = 0 \textbf{ alors } k \leftarrow k+3 \textbf{ sinon } k \leftarrow k+1}$
$(3 \leqslant k \leqslant 2027 \wedge k \bmod 4 = 3) \vee (1 \leqslant k \leqslant 2024 \wedge k \bmod 4 = 0)$
$\Rightarrow Q$

L'invariant de boucle suivant est correct mais moins fin :
a) $Q : 0 \leqslant k \leqslant 2028$
b) $Q \wedge k < 2026$
$\Rightarrow 0 \leqslant k \leqslant 2025$
$\xRightarrow{\textbf{si } k \bmod 2 = 0 \textbf{ alors } k \leftarrow k+3 \textbf{ sinon } k \leftarrow k+1} 1 \leqslant k \leqslant 2028$
$\Rightarrow Q$

Solution à l'exercice 4

Soit $B(x, p, n)$ le nombre défini pour tout réel x, tous entiers p et n tels que $0 \leqslant n \leqslant p$ par

$$B(x, p, n) = \sum_{j=0}^{n} \frac{p!}{j!(p-j)!} x^j.$$

Dans la suite vous devez donner des expressions de $B(x, p, n)$ pour certaines valeurs de p et n, puis construire un algorithme de calcul de $B(x, p, n)$ dans un modèle calculatoire où ne figurent pas les opérations élévation à la puissance et factorielle, puis porter cet algorithme en C.

Q13 Expression de $B(x, 4, 1)$.

$1 + 4x$

Q14 Expression de $B(x, 4, 3)$.

$1 + 4x + 6x^2 + 4x^3$

Q15 Identificateurs des données de l'algorithme.

x, p, n

Q16 Identificateurs des variables de l'algorithme.

k, t, s

Q17 Assertion d'entrée P.

$0 \leqslant n \leqslant p$

Q18 Assertion de sortie R.

$s = B(x, p, n)$

Q19 Assertion Q (futur invariant de boucle). *Conseil :* utilisez « B » avec de bons paramètres.

$$0 \leqslant k \leqslant n \wedge t = \frac{p!}{k!(p-k)!}x^k \wedge s = B(x,p,k)$$

Q20 Fin de l'induction : expression d'une condition (d'itération) C telle que $Q \wedge \neg C \Rightarrow R$.

$$k < n$$

Q21 Preuve que $Q \wedge \neg C \Rightarrow R$.

$$\begin{aligned} Q \wedge \neg C &\Rightarrow 0 \leqslant k \leqslant n \wedge s = B(x,p,k) \wedge k \geqslant n \\ &\Rightarrow \quad k = n \quad \wedge \quad \ldots\ldots \\ &\Rightarrow s = B(x,p,n) \\ &\Rightarrow R \end{aligned}$$

Q22 Induction (hérédité) : expression d'une action A' qui se termine et qui satisfait le schéma pré-post $Q \wedge C \xRightarrow{A'} Q$.

$$\begin{aligned} & k \leftarrow k+1 \\ & t \leftarrow t \cdot \frac{p-k+1}{k} \cdot x \\ & s \leftarrow s+t \end{aligned}$$

Q23 Preuve que $Q \wedge C \xRightarrow{A'} Q$.

$$\begin{aligned} & Q \wedge C \\ & \Longrightarrow \quad 0 \leqslant k \leqslant n \quad \wedge \quad t = \frac{p!}{k!(p-k)!}x^k \quad \wedge \quad s = B(x,p,k) \\ & \qquad \wedge\, k < n \\ & \Longrightarrow 0 \leqslant k \leqslant n-1 \wedge \quad \ldots\ldots\ldots\ldots \quad \wedge \quad \ldots\ldots\ldots \\ & \xRightarrow{k \leftarrow k+1} \quad 1 \leqslant k \leqslant n \quad \wedge\, t = \frac{p!}{(k-1)!(p-k+1)!}x^{k-1} \wedge s = B(x,p,k-1) \\ & \xRightarrow{t \leftarrow t \cdot \frac{p-k+1}{k} \cdot x} \quad \ldots\ldots \quad \wedge \quad t = \frac{p!}{k!(p-k)!}x^k \quad \wedge \quad \ldots\ldots\ldots \\ & \xRightarrow{s \leftarrow s+t} \quad \ldots\ldots \quad \wedge \quad \ldots\ldots\ldots\ldots \quad \wedge \quad s = B(x,p,k) \\ & \Longrightarrow Q \end{aligned}$$

Q24 Expression d'une quantité de contrôle pour la boucle associée au schéma pré-post $Q \wedge C \xRightarrow{A'} Q$.

$$k$$

Q25 Preuve que la quantité de contrôle annoncée en est bien une.

La suite des valeurs k est une suite entière, qui croit strictement à chaque itération de la boucle et qui est majorée par n.

Q26 Base (initialisation) : expression d'une action A qui se termine et qui satisfait le schéma pré-post $P \xRightarrow{A} Q$.

$$\begin{aligned} & k \leftarrow 0 \\ & t \leftarrow 1 \\ & s \leftarrow 1 \end{aligned}$$

Q27 Expression, sans la paire d'accolades englobantes, du corps d'une fonction C, de type résultant `double`, de paramètres `x` du type `double`, `p` et `n` du type `int`, qui renvoie $B(\texttt{x}, \texttt{p}, \texttt{n})$ lorsque ce nombre est défini et qui, sinon, donne à `errno` la valeur `EDOM` et renvoie une valeur nulle. Inutile de redonner l'invariant de boucle et la quantité de contrôle.

```
if (n < 0 || p < n) {
  errno = EDOM;
  return 0.0;
}
int k = 0;
double t = 1.0;
double s = 1.0;
while (k < n) {
  k += 1;
  t *= x * (p - k + 1) / k;
  s += t;
}
return s;
```

Algorithmique 1 : méthodologie de la programmation impérative
Contrôle continu n° 2

Date : 04-04-2025. Durée : 1 h 30. Documents, casques audio, oreillettes, calculatrices, mobiles, tablettes, portables, objets connectés, montres, trousses... interdites.

Commencez par renseigner TOUTES les cases « NOM Prénom numéro » qui figurent sur la liasse ; il y en a DEUX ; le NOM est à écrire en CAPITALES, pas le prénom ; le numéro est celui d'étudiant·e. Toute question attend une réponse dans le cadre situé immédiatement sous elle. En cas de manque de place : il sera écrit dans la case « VOIR COPIE À PART » ; la réponse, précédée de son numéro « Qn », sera donnée dans son intégralité sur une copie à part, fournie à la demande ; toute copie à part utilisée ne pourra l'être que pour un exercice donné. Ni l'espace figurant à l'intérieur des cadres ni les copies ne sauraient faire office de brouillon.

Les éléments de preuve que sont les INVARIANTS DE BOUCLE et les QUANTITÉS DE CONTRÔLES comptent pour entre le quart et la moitié des points accordés à chacune des questions.

NOM Prénom numéro :

Exercice 1 *(3 + 4 points.)*

Dans cet exercice, `element` désigne un type dont les valeurs peuvent être comparées deux à deux à l'aide des opérateurs binaires usuels : `==`, `!=`, `<`, `<=`, `>` et `>=`, et les tableaux ont pour composants des objets du type `element`.

Q1 Codez la fonction dont la déclaration est

```
size_t llcs(const element *a1, size_t n1, const element *a2, size_t n2);
```

qui, des tableaux repérés par `a1` et `a2`, de longueurs respectives `n1` et `n2`, renvoie la longueur de leur plus long SUFFIXE commun (*length of the longest common suffix*).

Q2 Codez la fonction dont la déclaration est

```
size_t llpr(const element *a, size_t n);
```

qui, du tableau repéré par `a`, de longueur `n`, renvoie la longueur du plus long PRÉFIXE constitué de la répétition de valeurs égales.

Macroconstantes et prototypes de fonctions standard C tels que présentés dans le cours :

```
#include <stdio.h>
NULL
FILE *fopen(const char *filename, const char *mode);
int fclose(FILE *stream);
EOF
int fgetc(FILE *stream);
int fputc(int c, FILE *stream);
int fscanf(FILE *stream, const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
int feof(FILE *stream);
int ferror(FILE *stream);
int fflush(FILE *stream);
int fseek(FILE *stream, long int offset, int whence);
SEEK_SET, SEEK_CUR, SEEK_END
long int ftell(FILE *stream);
void rewind(FILE *stream);
```

NOM Prénom numéro :

Exercice 2 *(4 + 4 + 5 points.)*

Dans cet exercice :

— `status` est l'alias d'un type énuméré comprenant, dans la liste de ses identificateurs de valeurs deux à deux distinctes, `UNMODIFIED`, `MODIFIED` et `DELETED` ;

— `record` est l'alias d'une structure dont l'un des champs est du type `status` et a pour identificateur `stts` ;

— les fonctions à coder dans la suite possèdent un premier paramètre qui est un pointeur de contrôleur associé à un fichier homogène sur le type `record`. Le fichier est supposé ouvert en mode mise à jour. Les fonctions ne doivent en aucun cas le fermer. Aucune hypothèse n'est faite quant au positionnement sur le fichier au début de l'exécution des fonctions. Aucune contrainte n'est imposée aux fonctions quant au positionnement sur le fichier à la fin de leurs exécutions.

Q3 Codez la fonction dont la déclaration est

```
int count_status(FILE *stream, status stts, long int *nptr);
```

qui tente de compter le nombre de composants du fichier associé à `stream` dont le champ `stts` a pour valeur celle du paramètre `stts`. En cas d'échec pour cause d'erreur survenue lors d'une opération sur le fichier, la fonction renvoie une valeur non nulle. En cas de succès, la fonction affecte à l'objet pointé par `nptr` le nombre de composants comptés et renvoie zéro.

Q4 Codez la fonction dont la déclaration est

```
int copy_non_deleted(FILE *src, const char *destfilename);
```

qui tente de CRÉER le fichier homogène sur le type `record` dont le nom est celui pointé par `destfilename` et d'y recopier les composants du fichier associé à `src` dont le champ `stts` N'A PAS la valeur `DELETED`. En cas d'échec pour cause d'erreur survenue lors d'une opération sur les fichiers, la fonction renvoie une valeur non nulle. En cas de succès, la fonction renvoie zéro.

Q5 Codez la fonction dont la déclaration est

```
int erase_modified(FILE *stream);
```

qui tente de modifier le champ `stts` des composants du fichier associé à `stream` dont la valeur est `MODIFIED` en fixant cette valeur à `UNMODIFIED`. En cas d'échec pour cause d'erreur survenue lors d'une opération sur le fichier, la fonction renvoie une valeur non nulle. En cas de succès, la fonction renvoie zéro.

(suite possible pour la réponse à la question Q5)

Corrigé

Solution à l'exercice 1

Dans cet exercice, `element` désigne un type dont les valeurs peuvent être comparées deux à deux à l'aide des opérateurs binaires usuels : `==`, `!=`, `<`, `<=`, `>` et `>=`, et les tableaux ont pour composants des objets du type `element`.

Q1 Codez la fonction dont la déclaration est

```
size_t llcs(const element *a1, size_t n1, const element *a2, size_t n2);
```

qui, des tableaux repérés par `a1` et `a2`, de longueurs respectives `n1` et `n2`, renvoie la longueur de leur plus long SUFFIXE commun (*length of the longest common suffix*).

```
size_t llcs(const element *a1, size_t n1, const element *a2, size_t n2) {
  size_t k = 0;
  //  IB : 0 <= k && k <= n1 && k <= n2
  //    && a1[n1 - k ... n1 - 1] == a2[n2 - k ... n2 - 1]
  //  QC : k
  while (k < n1 && k < n2 && a1[n1 - k - 1] == a2[n2 - k - 1]) {
    ++k;
  }
  return k;
}
```

Q2 Codez la fonction dont la déclaration est

```
size_t llpr(const element *a, size_t n);
```

qui, du tableau repéré par `a`, de longueur `n`, renvoie la longueur du plus long PRÉFIXE constitué de la répétition de valeurs égales.

```
size_t llpr(const element *a, size_t n) {
  if (n == 0) {
    return 0;
  }
  size_t k = 1;
  //  IB : 1 <= k && k <= n
  //    && a[0 ... k - 1] est composé de k répétitions de a[0]
  //  QC : k
  while (k < n && a[k] == a[0]) {
    ++k;
  }
  return k;
}
```

Solution à l'exercice 2

Dans cet exercice :

— `status` est l'alias d'un type énuméré comprenant, dans la liste de ses identificateurs de valeurs deux à deux distinctes, `UNMODIFIED`, `MODIFIED` et `DELETED` ;

— `record` est l'alias d'une structure dont l'un des champs est du type `status` et a pour identificateur `stts` ;

— les fonctions à coder dans la suite possèdent un premier paramètre qui est un pointeur de contrôleur associé à un fichier homogène sur le type `record`. Le fichier est supposé ouvert en mode mise à jour. Les fonctions ne doivent en aucun cas le fermer. Aucune hypothèse n'est faite quant au positionnement sur le fichier au début de l'exécution des fonctions. Aucune contrainte n'est imposée aux fonctions quant au positionnement sur le fichier à la fin de leurs exécutions.

Q3 Codez la fonction dont la déclaration est

```
int count_status(FILE *stream, status stts, long int *nptr);
```

qui tente de compter le nombre de composants du fichier associé à `stream` dont le champ `stts` a pour valeur celle du paramètre `stts`. En cas d'échec pour cause d'erreur survenue lors d'une opération sur le fichier, la fonction renvoie une valeur non nulle. En cas de succès, la fonction affecte à l'objet pointé par `nptr` le nombre de composants comptés et renvoie zéro.

```
int count_status(FILE *stream, status stts, long int *nptr) {
  rewind(stream);
  record x;
  *nptr = 0;
  //  sans tenir compte des derniers effets de l'évaluation de l'ECB
  //  IB : *nptr == nombre de composants lus sur stream dont le champ stts a
  //    pour valeur celle du paramètre stts
  //  QC : nombre d'appels à fread
  while (fread(&x, sizeof(record), 1, stream) == 1) {
    *nptr += (x.stts == stts);
  }
  if (!feof(stream)) {
    return -1;
  }
  return 0;
}
```

Q4 Codez la fonction dont la déclaration est

```
int copy_non_deleted(FILE *src, const char *destfilename);
```

qui tente de CRÉER le fichier homogène sur le type `record` dont le nom est celui pointé par `destfilename` et d'y recopier les composants du fichier associé à `src` dont le champ `stts` N'A PAS la valeur `DELETED`. En cas d'échec pour cause d'erreur survenue lors d'une opération sur les fichiers, la fonction renvoie une valeur non nulle. En cas de succès, la fonction renvoie zéro.

```
int copy_non_deleted(FILE *src, const char *destfilename) {
  FILE *dest = fopen(destfilename, "wbx");
  if (dest == nullptr) {
    return -1;
  }
  rewind(src);
  record x;
  //  sans tenir compte des derniers effets de l'évaluation de l'ECB
  //  IB : tous les composants lus sur src dont le champ stts n'a pas la valeur
  //        DELETED ont été écrits sur dest
  //  QC : nombre d'appels à fread
  while (fread(&x, sizeof(record), 1, src) == 1
      && (x.stts == DELETED || fwrite(&x, sizeof(record), 1, dest) == 1)) {
  }
  int r = 0;
  if (!feof(src)) {
    r = -1;
  }
  if (fclose(dest) != 0) {
    r = -1;
  }
  return r;
}
```

Q5 Codez la fonction dont la déclaration est

```
int erase_modified(FILE *stream);
```

qui tente de modifier le champ `stts` des composants du fichier associé à `stream` dont la valeur est `MODIFIED` en fixant cette valeur à `UNMODIFIED`. En cas d'échec pour cause d'erreur survenue lors d'une opération sur le fichier, la fonction renvoie une valeur non nulle. En cas de succès, la fonction renvoie zéro.

```
int erase_modified(FILE *stream) {
  rewind(stream);
  //  sans tenir compte des derniers effets de l'évaluation de l'ECB
  //  IB : la valeur du champ stts des composants lus sur stream a été fixé à
  //    UNMODIFED pour tous ceux pour laquelle elle était initialement MODIFIED
  //  QC : nombre d'appels à erase_modified_aux
  while (erase_modified_aux(stream) == 0) {
  }
  if (!feof(stream)) {
    return -1;
  }
  return 0;
}
```

avec :

```
//  erase_modified_aux : tente de lire le composant courant du fichier homogène
//    de record associé au flot stream ouvert en mise à jour et, si son champ
//    stts a pour valeur est MODIFIED, de fixer cette valeur à UNMODIFIED. Si
//    l'opération réussit, le repositionnement sur le flot est assuré exactement
//    comme si une simple lecture avait eu lieu. Renvoie zéro en cas succès, une
//    valeur non nulle en cas d'échec.
int erase_modified_aux(FILE *stream) {
  record x;
  if (fread(&x, sizeof(record), 1, stream) != 1) {
    return -1;
  }
  if (x.stts != MODIFIED) {
    return 0;
  }
  x.stts = UNMODIFIED;
  if (fseek(stream, -(long int) sizeof(record), SEEK_CUR) != 0
      || fwrite(&x, sizeof(record), 1, stream) != 1
      || fseek(stream, 0, SEEK_CUR) != 0) {
    return -1;
  }
  return 0;
}
```